# Universidad Nacional de Tucumán
# Facultad de Ciencias Exactas y Tecnología

Carro, Joaquín Manuel Enrique

2025

Proyecto final, Ingeniería de Software II

# Test Driven Development (TDD)

## 1. Introducción

Este trabajo se centrará en el análisis conceptual del Test-Driven Development como práctica de desarrollo de software dentro de la ingeniería de software. Abarcará su funcionamiento, su clasificación como modelo prescriptivo o descriptivo, su relación con metodologías ágiles y su impacto en la calidad del software. Relacionándolo así con distintos conceptos desarrollados en clase. También se realizará una comparación con otros modelos de desarrollo, buscando establecer si está es una buena metodología a seguir para el desarrollo de software.

## 2. Fundamentación

El Test Driven Development (TDD) o el Desarrollo Guiado por Pruebas es una técnica de ingeniería para desarrollar software que se basa en primero diseñar pruebas, acordes a los requisitos deseado, a partir de las cuales se construirá el programa final.

Esta técnica fue desarrollada por Kent Beck con el objetivo de conseguir código que resultase más tolerante al cambio, robusto, seguro, más barato de mantener y de rápido desarrollo.

Cuando se trabaja bajo la metodología TDD, se impulsa el desarrollo de software mediante pruebas automatizadas, siguiendo dos simples reglas: solo escribir código nuevo cuando una prueba automática falla y eliminar los duplicados. Para cumplir con estas reglas a la hora de programar, se sigue un ciclo conocido como “Red-Green-Refactor”, el cual consiste en:

- **Red**: realizar un test que falle, solo el test, sin su implementación. Generalmente los test que aquí se realizan son unitarios.

- **Green**: hacer que el test funcione rápido, con el mínimo código posible, sin importar si se cometen errores para conseguirlo.
- **Refactor**: una vez que el test funciona, examinarlo buscando eliminar duplicaciones o aplicar mejoras.

Seguir este ciclo a la hora de programar tiene dos beneficios inmediatos, se obtiene un código con una buena cobertura de fallos y, además, permite establecer la funcionalidad de cada componente específica, según la prueba diseñada.

Podemos concluir entonces que cuando desarrollamos software bajo la práctica Test Driven Development, estamos desarrollando un código con gran adaptabilidad a los cambios y cuya documentación relacionada al comportamiento del sistema, resulta simple por provenir de pequeños test específicos que satisfacen comportamientos concretos, pero a su vez, también resulta ser bastante completa ya que se especifica el funcionamiento de cada función y sus escenarios de aplicación, como también que tipos de datos recibe y produce.

Ejemplificaremos el funcionamiento del ciclo Red-Green-Refactor con los siguientes ejercicios:

a. Se desea obtener una función que sea capaz de convertir un determinado monto en peso argentino, a su equivalente a dólares, conociendo el tipo de cambio correspondiente. Además, tendremos en cuenta que la función no debe aceptar valores negativos para el monto de peso.

   Función básica: supondremos que se quieren convertir 150000 pesos a dólares con un tipo de cambio de 1500

   - Red: test que falle

```
PRUEBA: conversión
resultado ← pesos_a_dolares(150000,1500)
VERIFICAR resultado = 100
```

Esta prueba fallara, ya que la función pesos_a_dolares no existe todavía.

- Green: hacer que el test funcione de manera simple.

```
FUNCION: pesos_a_dolares(pesos, cambio)

RETORNAR pesos/cambio
```

Realizamos la prueba ahora y está será exitosa.

- Refactor: como hasta acá el caso es muy sencillo, no requiere de ajustes.

Función valores negativos: supondremos ahora que se quiere convertir un monto en pesos de -1500 con un cambio de 1500

- Red: prueba que falle.

```
PRUEBA: valor_negativo

resultado ← pesos_a_dolares(-1500,1500)

VERIFICAR produzca ERROR
```

La prueba falla, pues no existe la función pesos_negativos_a_dolares.

- Green: hacer funcionar a la prueba.

```
FUNCION: pesos_a_dolares(pesos_negativos, cambio)

SI pesos < 0 ENTONCES

        RETORNAR ERROR

SINO

        RETORNAR pesos/cambio
```

Se ejecuta la prueba y se pasa exitosamente.

- Refactor: en este caso como tal, no hay mejoras que se puedan realizar sobre el código, lo que si se hace es quedarnos únicamente con la segunda función

realizada, de esta forma evitamos duplicaciones y la segunda función contiene a la primera, por lo tanto, las pruebas seguirán pasando correctamente.

b. Se desea una determinar si un número de documento especifico forma parte de una lista de proporcionada previamente. Se debe tener en cuenta que los números de documentos son tratados como cadenas de caracteres de 8 dígitos.

Prueba básica para el caso en el que DNI buscado se encuentre en la lista:

- Red:

```
PRUEBA: verificar_existe_dni
lista ← ['12345678','87654321','11111111','88888888']
resultado ← existe_dni('12345678', lista)
VERIFICAR resultado = TRUE
```

- Green:

```
FUNCION: existe_dni(dni, lista)
PARA cada elemento EN lista HACER
        SI elemento = dni ENTONCES
                RETORNAR TRUE
        SINO
                RETORNAR FALSE
```

Ahora procedemos a realizar la prueba para el caso en el que el DNI no se encuentre en la lista:

- Red:

```
PRUEBA: verificar_no_existe_dni
lista ← ['12345678','87654321','11111111','88888888']
resultado ← existe_dni('22222222', lista)
VERIFICAR resultado = FALSE
```

La función existe_dni ya abarca está prueba y la pasa exitosamente, por lo que no es necesario realizar más código.

Por último, vamos a considerar el caso en el que el número de documento que se desea buscar, no tenga el formato correspondiente, es decir, que no sea de ocho caracteres:

- Red:

```
PRUEBA: verificar_formato_valido
lista ← ['12345678','87654321','11111111','88888888']
resultado ← existe_dni('1234', lista)
VERIFICAR produzca ERROR
```

- Green:

```
FUNCION: existe_dni(dni, lista)
SI longitud(dni) != 8 entonces
        RETORNAR ERROR
SINO
        PARA cada elemento EN lista HACER
                SI elemento = dni ENTONCES
                        RETORNAR TRUE
                SINO
                        RETORNAR FALSE
```

- REFACTOR: en este caso una buena práctica de refactorización seria separar las responsabilidades de la función en funciones diferentes.

```
FUNCION: existe_dni(dni, lista)
dni_valido(dni)
PARA cada elemento EN lista HACER
        SI elemento = dni ENTONCES
                RETORNAR TRUE
        SINO
        RETORNAR FALSE
```

```
FUNCION: dni_valido (dni)
SI longitud(dni) != 8 ENTONCES
        RETORNAR ERROR
```

## 3. TDD y las metodologías ágiles

Si hablamos de enfoques para el desarrollo de software centrados en la capacidad de adaptarse a cambios frecuentes y entregar avances en forma constante, entonces debemos considerar la relevancia de las metodologías ágiles.

Aunque en apariencia TDD podría interpretarse simplemente como una técnica de “Test First Development”, su verdadero potencial se manifiesta cuando se integra con prácticas propias del desarrollo ágil, al combinarse con metodologías como “Extreme Programming” o “Scrum”, TDD deja de centrarse únicamente en la verificación del software y pasa a desempeñar un papel importante en el proceso de diseño y construcción incremental del sistema.

A partir de esta integración, el proceso de desarrollo puede organizarse en una serie de pasos iterativos que permiten implementar cada funcionalidad de forma gradual, validando continuamente su comportamiento mediante pruebas automatizadas:

1) **Definición de la historia de usuario**: el cliente o Product Owner describe las funcionalidades deseadas del sistema.
2) **Definición de requisitos**: el equipo de desarrollo, en colaboración con el cliente, extraen los requisitos que debe cumplir el sistema para considerar que es correcto.
3) **Selección de un requisito**: se escoge el requisito más simple con el fin de implementarlo.
4) **Construcción de la prueba unitaria (Red)**: se diseña una prueba que falle, que satisfaga el requisito seleccionado.
5) **Implementación del código (Green)**: se desarrolla el código mínimo necesario que permita que la prueba sea superada.
6) **Ejecución pruebas automatizadas**: se ejecutan el conjunto de pruebas existentes para asegurar que la nueva funcionalidad no afecta a otros elementos del sistema previamente implementados.
7) **Refactorización del código (Refactor)**: cuando se superan las pruebas satisfactoriamente, se mejora y optimiza el código.
8) **Verificación**: tras la refactorización, se vuelven a ejecutar las pruebas automatizadas para garantizar que el comportamiento del sistema se mantiene intacto.
9) **Repetición**: se repite el proceso con todos los requisitos restantes.

En este proceso, cada paso está bien definido, volviéndose sencillos y rápido tras la repetición constante del mismo. Resulta fácil observar, como TDD se ajusta perfectamente a los 12 principios que guían las metodologías ágiles.

## 4. Impacto en la calidad

En todo proyecto de desarrollo de software, sin importar cuales sean las metodologías utilizadas, siempre uno de los principales objetivos es construir un sistema con una buena calidad, y la metodología TDD no es la excepción.

Establecer el nivel de calidad o confianza de un software nunca es fácil, y si bien Test Driven Development no define un sistema formal de gestión de calidad, actúa como un mecanismo de aseguramiento de calidad incorporado al proceso de desarrollo. Podemos decir que, por su forma de llevarse a cabo, TDD cumple con las actividades de planificación, aseguramiento y control de la calidad.

TDD gestiona la calidad mediante la prevención de defectos de forma temprana, integrando pruebas automatizadas desde el inicio del desarrollo, en lugar de detectar defectos al final del proceso. De esta forma, busca garantizar continuamente la validación de los test. Además, facilita la mejora del diseño mediante la refactorización que forma parte del proceso mismo.

Si bien, ningún estudio demuestra categóricamente las diferencias a favor en productividad o calidad entre utilizar TDD u otras opciones, tampoco ningún estudio demuestra lo contrario. Sin embargo, las experiencias de los equipos al trabajar utilizando TDD demuestra que suelen estar relajados y con confianza en cada parte del proyecto, y que a los clientes les emociona la constante de avances que se le proporciona.

## 5. Beneficios de TDD

- **Claridad en los requisitos:** permite establecer una compresión clara de cada requisito especifico del programa antes de comenzar a trabajar.
- **Fomenta un desarrollo ágil**: a largo plazo, TDD se vuelve muy eficiente gracias al permitir reducir el tiempo dedicado a depurar y corregir errores.
- **Mayor confianza**: los desarrolladores consiguen un gran nivel de confianza tanto en su código como en las pruebas realizadas sobre el mismo.
- **Estructura en el desarrollo**: aporta una estructura clara sobre cómo debe ser el proceso de desarrollo.
- **Productividad mejorada**: una vez acostumbrado a utilizar TDD, permite automatizar grandes proyectos en pequeños pasos alcanzables.
- **Documentación clara**: al realizar pruebas sobre todo el código desarrollado, la documentación y especificaciones resulta ser muy sólidas, validando las características y requisitos deseados.
- **Rapidez en la retroalimentación**: el ciclo de realimentación en las decisiones de diseño es obviamente corto, permitiendo desarrollar ideas en interfaces funcionales en poco tiempo.
- **Mantenibilidad y escalabilidad**: TDD reduce los riesgos asociados al mantenimiento y la escalabilidad del software, ya que un código bien diseñado y probado es más fácil de mantener y escalar.
- **Satisfacción del cliente**: el cliente puede percibir avances constantes y funcionales, que se adaptan correctamente a sus requerimientos.

## 6. Desafíos de TDD

- **Curva de aprendizaje inicial**: realizar códigos que resulten efectivos requiere de mucha práctica y puede resultar realmente difícil para aquellas personas que no estén acostumbradas.
- **Mayor tiempo necesario**: la etapa inicial hasta conseguir test funcionales ralentiza el proceso. También se debe dedicar un tiempo considerable en configurar y mantener entornos de prueba que resulten adecuados para el código.
- **Disminución eficiencia**: aunque se ha demostrado que el TDD mejora la productividad, puede retrasar el desarrollo del proyecto, ya que añade más pasos a la creación e implementación de cada nueva característica.
- **Mayor volumen de código**: como se debe desarrollar tanto el código funcional, como así también los test iniciales, esto resulta en una gran cantidad general de código.
- **Mucho esfuerzo en el mantenimiento**: mantener los test actualizados, los cuales crecen proporcionalmente junto con el proyecto, considerando los cambios en el código por la refactorización y el agregado de nuevos requisitos, puede convertirse en una tarea desafiante.
- **Integración con sistemas existentes**: Aplicar TDD en sistemas legados o en proyectos en marcha puede ser un reto ya que no han sido diseñados con la mentalidad diseñar a partir de las pruebas.
- **Descuido del sistema general**: aunque el TDD fomenta la sencillez del código y mejora el diseño, centrarse demasiado en los componentes individuales puede llevar a un código general poco unificado.

## 7. TDD y los modelos de desarrollo

El Test Driven Development puede considerarse un enfoque prescriptivo, ya que establece una secuencia específica de actividades que deben seguirse durante el desarrollo del software. Como hemos analizado, TDD prescribe un procedimiento claro y repetible basado en el ciclo Red–Green–Refactor, pero sin entrar en especificaciones sobre la manera en la que se debe realizar el desarrollo del software.

Aunque el TDD no constituye un modelo completo de proceso de desarrollo de software, resulta útil compararlo con algunos de los modelos de proceso estudiados en el curso. Esta comparación permite analizar cómo difieren estos enfoques en aspectos como la organización del desarrollo, la forma en que progresa el proyecto y el rol que cumplen las pruebas dentro del proceso de construcción del software.

| **Modelo** | **TDD** | **Secuencial** | **Iterativo** | **Incremental** |
|---|---|---|---|---|
| **Método** | Desarrollo guiado por pruebas. Primero se escriben las pruebas y luego el código | Proceso lineal donde cada fase es completada antes de comenzar la siguiente | Desarrollo mediante ciclos repetidos mejorando una versión inicial | Se construye y entrega el producto por partes que suponen un incremento |
| **Desarrollo** | Iterativo a pequeña escala, centrado en pruebas unitarias y refactorización constante | El resultado de una etapa es la entrada de la siguiente | Mejoras progresivas, pero no siempre se añaden nuevas funcionalidades | En cada incremento se añaden nuevas funcionalidades |
| **Objetivo** | Asegurar calidad, reducir errores y mejorar el diseño con pruebas automatizadas | Organizar el desarrollo de forma estructurada y ordenada | Refinar y mejorar el producto existente hasta obtener el adecuado | Entregar valor funcional de forma temprana y progresiva |
| **Progreso** | Se entregan pequeños avances basados en funcionalidades especificas | Entrega el producto completo una vez finalizado | Se entrega un producto completo al inicio y se mejora gradualmente | Se construyen parte del producto en cada ciclo hasta completarlo |

A continuación, buscaremos dejar más en claro las diferencias entre estos modelos mediante un ejercicio: Se desea implementar una calculadora que sea capaz de sumar o restar dos números.

- TDD: primero las pruebas que fallen, luego hacerlas funcionar y por último refactorizar.

```
PRUEBA: sumar_dos_numeros
resultado ← suma(2,3)
VERIFICAR resultado = 5
```

```
FUNCION: suma(a,b)
resultado ← a + b
RETORNAR resultado
```

```
FUNCION: suma(a,b)
RETORNAR a + b
```

```
PRUEBA: restar_dos_numeros
resultado ← resta(3,2)
VERIFICAR resultado = 1
```

```
FUNCION: resta(a,b)
resultado ← a - b
RETORNAR resultado
```

```
FUNCION: resta(a,b)
RETORNAR a - b
```

- Secuencial: proceso paso a paso, se entrega el producto completo al final.

4. Análisis: el sistema debe permitir:
    - Sumar dos números
    - Restar dos números

3. Diseño:
    - FUNCION suma(a,b)
    - FUNCION resta(a,b)

2. Codificación:

FUNCION suma(a,b)

resultado ← a + b

RETORNAR resultado

FUNCION resta(a,b)

resultado ← a - b

RETORNAR resultado

1. Pruebas:

Prueba sumar:

VERIFICAR suma(2,3) = 5

Prueba restar:

VERIFICAR resta(3,2) = 1

- Iterativo: se entrega un producto completo que se mejora con cada iteración.

Iteración 1:

FUNCION suma(a,b)

resultado ← a + b

RETORNAR resultado

FUNCION resta(a,b)

resultado ← a - b

RETORNAR resultado

Iteración 2:

FUNCION suma(a,b)

RETORNAR a + b

FUNCION resta(a,b)

RETORNAR a - b

- Incremental: el producto se construye por partes que se entregan en cada incremento

Incremento 1:

FUNCION: suma(a,b)

RETORNAR a + b

Incremento 2:

FUNCION: resta(a,b)

RETORNAR a - b

## 8. Conclusiones

Considerando todo lo analizado en este trabajo, podemos concluir que al hacer uso de la metodología Test Driven Development, el elemento que cobra más relevancia son las pruebas o test, con las cuales por medio de la práctica del ciclo RED-GREEN-REFACTOR nos permite obtener un código simple, funcional y que satisfaga nuestros requerimientos.

Aunque esta metodología puede resultar un poco complicada al principio para aquellos que no están acostumbrados a su uso e implique una mayor cantidad volumen de código en comparación a otros métodos, trabajar con ella le aporta un gran nivel de confianza al proyecto, gracias a su detección temprana de errores y su capacidad de adaptación a los cambios continuos. También nos permite resaltar con facilidad los requisitos del sistema y proporciona una documentación clara sobre el funcionamiento del mismo.

Por lo tanto, es adecuado concluir que el uso de TDD es conveniente en los siguientes contextos:

- El equipo de trabajo tiene cierta experiencia con testing.
- El proyecto es de larga duración con ampliaciones o modificaciones continuas.
- La calidad y confiabilidad sean elementos críticos del proyecto.

Bibliografia


- Kent, B. (2002). Test-Driven Development by Example.
- Gómez, M. (). TDD (Test Driven Development). Desarrollo diriguido por pruebas. https://softwarecrafters.io/javascript/tdd-test-driven-development
- Herranz, J, I. (2023). TDD como metodologpia de diseño de software. https://www.paradigmadigital.com/dev/tdd-como-metodologia-de-diseno-de-software/
- Schneider, J & Smalley, I. (). ¿Qué es el desarrollo basado en pruebas (TDD)?. https://www.ibm.com/es-es/think/topics/test-driven-development
- Codurance. (). TDD – Desarrollo guiado por pruebas. ¿Cómo aplicar TDD en tus proyectos de desarrollo de software?. https://www.codurance.com/es/guia-completa-desarrollo-guiado-por-pruebas-tdd